EdgeVault

AI-Assisted Trading Journal

AI-101 Assignment Report
Panaversity — Agentic AI Architect Program

Student: RaiedOP
Tool Used: Claude Sonnet (Anthropic)
Date: June 2026

1. Project Overview

Project Title	EdgeVault — Professional Trading Journal
App Category	Finance / Productivity / Self-Improvement Tool
Platform	Browser-based single HTML file (Desktop + Mobile)
AI Tool Used	Claude Sonnet 4.6 by Anthropic (claude.ai)
Language	HTML, CSS, JavaScript (vanilla, no frameworks)
Assignment	AI-101 — Build an app of your choice using AI assistance

App Idea

EdgeVault is a fully self-contained, browser-based trading journal for retail forex and gold traders. The idea came from a personal need — most free journals are too basic, subscription-based, or require cloud sign-up. EdgeVault runs entirely from a single HTML file with no server, no database, and no internet dependency after the first load. It stores data locally and can export a self-contained backup file with all trades embedded inside it.

Purpose of the Project

To give retail traders a professional-grade journaling tool that is free, private, portable, and works on any device. Traders need to track trades, analyse performance, manage emotions, follow prop firm rules, and review setups — all in one place. EdgeVault solves this without subscriptions, data collection, or internet dependency.

2. AI-Assisted Development Process

The entire application was built through iterative conversations with Claude Sonnet on claude.ai. No code was written manually — all logic, styling, and features were generated and refined through natural language prompting. The project went through approximately 6 major version iterations (v1 through v4+), each adding significant functionality.

Initial Prompt

The project started with this opening prompt:

```
"Build me a trading journal as a single HTML file. It should let me log trades
with entry, stop loss, take profit, lot size, result, and notes. Dark theme.
Make it look professional."
```

This gave Claude enough context to understand the domain (trading), format (single HTML), visual direction (dark professional), and minimum data fields. The output was a working v1 within a single response.

Prompt Engineering Techniques Used

Specificity with Examples

Rather than "add analytics", prompts specified exact metrics: "Add win rate, profit factor, max drawdown, average R:R, and expectancy." Concrete requirements produce concrete outputs.

Incremental Building

Each prompt built on the previous: "Keep everything from v3, now add a pending result state." This prevented rewrites and allowed feature accumulation without regressions.

Problem-First Prompting

Issues were described as observed symptoms, not code: "Losses show as positive numbers in EURUSD — the minus sign is missing." Claude diagnosed the root cause correctly each time.

Consolidation Prompts

"Here are 5 issues I found — fix all of them in one go." This reduced back-and-forth and kept the file clean rather than accumulating patch-on-patch edits.

Clarification Before Coding

For complex features: "tell me your opinion first, no need to code anything." This ensured the approach was agreed before implementation, avoiding wasted effort.

Domain-Specific Terminology

Trading terms — pip, lot size, R:R, prop firm, session, drawdown — were used deliberately so Claude applied domain knowledge rather than generic assumptions.

3. Prompt Iterations & Development Stages

v1 — Foundation

Prompt	`"Build a trading journal as a single HTML file with dark theme. Fields: entry, SL, TP, lots, result, notes."`
Outcome	Basic trade log table, add trade form, dark styling. Pure HTML/CSS/JS, no frameworks.

v2 — Psychology & Analytics

Prompt	`"Add emotion tracking, plan following toggle, confidence/stress/sleep sliders, win rate, profit factor, and equity curve chart."`
Outcome	Psychology section with emotion and violation tags. Dashboard KPI cards. Chart.js equity curve.

v3 — Prop Firm & Position Sizer

Prompt	`"Add a prop firm tracker with daily loss limit and target progress. Also add a position size calculator that auto-fills from account balance."`
Outcome	Prop Tracker page. Position Size Calculator with per-instrument pip configs. Payout calculator.

v4 — Pending Trades & Memory

Prompt	`"Add a Pending result so trades can be logged when placed, then updated later. Also the app loses all data when closed — fix this."`
Outcome	Pending result state with flag button to close trades. localStorage auto-save. Self-contained Save File export.

v4+ — Mobile & Math Fixes

Prompt	`"The app is zoomed out on mobile and overlapping. Make it fully responsive. Also fix pip values for cross pairs and oil."`
Outcome	Full mobile layout: bottom tab bar, swipeable table, single-column forms, bottom-sheet modals. Fixed 15+ pip configs.

v4+ — Result UX

Prompt	`"Add TP/SL/BE/Manual quick-select buttons and auto-calculate PnL and result automatically when exit is selected."`
Outcome	Quick-select buttons in both Add Trade and Set Result popup. Auto PnL with correct sign. Auto result from TP/SL levels.

4. What Worked Well

- Claude understood trading domain terminology without extra explanation — pip, lot size, R:R, drawdown, and session were all used correctly from the first response.
- Single-file architecture worked perfectly. The entire application lives in one HTML file that can be emailed, shared, or opened on any device.
- Dark cyberpunk UI aesthetic was generated and maintained consistently across all 6 versions with no style drift.
- Incremental prompting kept all previous features intact while adding new ones. No major regressions throughout the build.
- Describing bugs as observed symptoms ("losses show as positive") was more effective than trying to explain code locations. Claude diagnosed root causes accurately each time.
- The localStorage + embedded seed data approach for persistence was an elegant solution generated in one response — auto-saves locally AND travels with the exported file.
- Mobile layout conversion was clean — bottom tab bar, scrollable trade log, single-column forms, and bottom-sheet modals all generated correctly in one pass.
- Chart.js integration for equity curve and daily PnL charts worked on the first attempt with no corrections needed.

5. Bugs, Challenges & Solutions

Data not persisting on close

Problem	The initial storage used window.storage (Claude artifact API) which only works inside claude.ai — not in a local file. All data was lost on every close.
Solution	Replaced with localStorage as primary storage. Added a "Save File" export that embeds all trade data as JSON seed inside the HTML, making the file fully self-contained and portable.

Loss PnL showing as positive

Problem	In the trade log, negative PnL like -50 displayed as "$50" — minus sign was missing. Colour was correct (red) but the number appeared positive.
Solution	Found a one-character bug: pnl>=0?"+":"" was missing the minus. Fixed to pnl>=0?"+":"-" so losses now correctly display as "-$50".

Wrong pip values for cross pairs and oil

Problem	All pairs were set to a flat $10 pip value. This is only correct for USD-quoted pairs. USOIL had pip value of $1 when it should be $10 for a 1,000-barrel contract — a 10x error.
Solution	Audited all 50+ instruments. Fixed USDCAD (~$7.40), USDCHF (~$11.10), all cross pairs (EURGBP, EURAUD, GBPAUD etc.) and fixed USOIL/UKOIL from v:1 to v:10.

Mobile layout zoomed out and unreadable

Problem	Without a viewport meta tag, mobile browsers apply default zoom-out to fit the desktop layout. The sidebar also occupied full screen width leaving no room for content.
Solution	Added the viewport meta tag. Converted sidebar to a fixed bottom tab bar. Made trade log scroll horizontally. Collapsed forms to single column. Set inputs to 16px to prevent iOS auto-zoom.

Icons not showing in local file

Problem	All icon buttons (edit, delete, flag) were invisible when opened as a local file. The amber pending flag button was completely hidden.
Solution	The Tabler Icons CDN link was missing from the file. Added the @tabler/icons-webfont stylesheet link at the top of the HTML.

Screenshot quality unreadable

Problem	Compression was set to max 280px at 42% JPEG quality, making price levels and annotations completely unreadable in the gallery.
Solution	Increased max resolution to 1400px and JPEG quality to 88%. Screenshots are now sharp enough to read annotations clearly.

Infinity symbol showing as garbled text

Problem	The profit factor displayed as "a^z" (garbled bytes) instead of the infinity symbol. Encoding mismatch between the file and browser.
Solution	Added to the document head, fixing correct character rendering across all browsers.

6. Features Added & Improved

Feature	Status	Details
Trade Log	**Done**	Date, pair, direction, entry/SL/TP, lots, R:R, PnL, result, grade, screenshots
Dashboard KPIs	**Done**	Win rate, profit factor, max drawdown, avg R:R, expectancy, streak, best session
Equity Curve	**Done**	Chart.js line chart showing cumulative PnL over time
Psychology Tracker	**Done**	Emotion tags, violation tags, confidence/stress/sleep sliders, plan-follow toggle
Prop Firm Tracker	**Done**	Daily loss limit, profit target progress, drawdown rules per firm
Position Size Calculator	**Done**	Auto lot size from balance, risk %, entry and SL with per-instrument pip config
Payout Calculator	**Done**	Estimates prop firm payout based on profit split percentage
Screenshot Gallery	**Done**	Up to 6 screenshots per trade at 1400px / 88% JPEG quality
Pending Result State	**Done**	Trades logged when placed, closed later with TP/SL/BE/Manual quick-select
Auto PnL Calculation	**Done**	PnL auto-fills on exit selection with correct sign for Buy and Sell
Auto Result Selection	**Done**	Result auto-picks Win/Loss/Break Even based on exit vs TP/SL levels
localStorage Persistence	**Done**	All data auto-saves — survives browser close and reopen at same file path
Self-Contained Export	**Done**	Save File downloads HTML with all data embedded — portable across devices
Mobile Responsive Layout	**Done**	Bottom tab bar, horizontal table scroll, single-column forms, bottom-sheet modals
Instrument Pip Config	**Improved**	Fixed 15+ pairs: cross pairs, USD-base pairs, and oil contracts
Auto R:R Display	**Improved**	R:R calculated from entry/SL/TP on the fly even if not manually entered
Analytics Page	**Done**	Session breakdown, setup win rates, emotion performance, pair analysis
Trade Calendar	**Done**	Monthly calendar view showing daily PnL with colour coding
CSV Export	**Done**	One-click export of all trades to spreadsheet-compatible CSV

7. Final Result

EdgeVault v4 is a fully functional, professional-grade trading journal delivered as a single HTML file of approximately 700 lines. It requires no installation, no server, no internet connection after first load, and no subscription. It runs in any modern browser on desktop or mobile.

Technical Summary

File Size	~120KB single HTML file
Dependencies	Chart.js (CDN), Tabler Icons (CDN) — loaded on first open only
Storage	localStorage + embedded seed data in HTML export
Instruments	50+ forex pairs, metals, indices, crypto, oil with accurate pip configs
Mobile Support	iPhone, Android, any screen — responsive CSS media query at 768px breakpoint
Iterations	6 major versions, 30+ individual feature and fix sessions with Claude
Lines of Code	~700 HTML + CSS + JS (inline, no build tools required)

Does it Solve a Real-World Problem?

Yes. EdgeVault directly solves a genuine problem in the retail trading community. Most traders use spreadsheets (limited, no psychology tracking), paid SaaS journals ($20-50/month), or nothing at all. EdgeVault provides a complete alternative:

- Complete trade logging with all fields a professional trader needs
- Psychology tracking — the #1 factor most traders ignore but that determines long-term profitability
- Prop firm compliance tracking — critical for the growing funded trader community
- Accurate position sizing — the pip value fix prevented traders from over/under-sizing by up to 10x on oil pairs
- Zero cost, zero data collection, zero internet dependency after first load
- Mobile-ready — traders can log trades from their phone the moment they place them
- Portable — the Save File export means the journal travels across devices with all data intact

Conclusion: EdgeVault solves a real-world problem and delivers a tool that rivals paid alternatives — built entirely through AI-assisted prompt engineering with zero manual code written. This demonstrates the practical power of structured prompting to produce production-quality software in a domain-specific context.

This project was built as part of the Panaversity AI-101 course — Agentic AI Architect Program.
All code generated using Claude Sonnet 4.6 (Anthropic) — zero manual code written.